UNIVERSITY OF CAMBRIDGE
Department of Computer Science and Technology

May 15, 2026

Efficient Coinductives through State-Machine Corecursors

William Sørensen

Gonville & Caius College

Submitted in partial fulfilment of the requirements for the Computer Science Tripos, Part II

Declaration

I, William Sørensen of Gonville & Caius College, being a candidate for the course, hereby declare that this report and the work described in it are my own work, unaided except as may be specified below, and that the report does not contain material that has already been used to any substantial extent for a comparable purpose.
Signed: William Sørensen
Date: May 15, 2026

Proforma

Word count: 5577

Acknowledgements

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aeque doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguique possit, augeri amplificarique non possit.

Contents

1 Introduction ... 9
1.1 Use cases ... 1
1.1.1 Infinite datastructures ... 1
1.1.2 Compiler semantics ... 1
1.1.3 Safe webservers ... 1
2 Preparation ... 2
2.1 Dependent type theory ... 2
2.2 Lean ... 2
2.3 Polynomial functors ... 4
2.3.1 Common polynomials ... 4
2.4 Recursion and Inductives ... 5
2.5 Corecursion and coinductives ... 5
2.5.1 In OCaml ... 5
2.5.2 Streams ... 6
2.5.3 The M-type ... 6
2.5.4 State-machine encoding ... 7
2.6 Inductive predicate ... 8
2.7 Coinductive predicate ... 8
2.7.1 Bisimilarity ... 8
2.8 Methodology ... 9
2.9 Tools used ... 9
2.10 Requirement Analysis ... 10
2.10.1 State machine encoding (Core) ... 10
2.10.2 Non-termination-monad (Core) ... 10
2.10.3 ABI Type (Extention) ... 10
2.10.4 Interaction Trees (Extention) ... 10
2.10.5 Futumorphic productivity (Extention) ... 11
3 Implementation ... 12
3.1 Repository overview ... 12
3.2 Common ... 13
3.2.1 Transiteration ... 13
3.2.2 Expanding the progressive approximation theory ... 13
3.3 Stream implementation ... 14
3.3.1 SME ... 14
3.3.2 PA ... 15
3.3.3 Proving the equivalence ... 15
3.4 State machine encoding of M-Types ... 16
3.4.1 PreM ... 16
3.4.2 Writing for performance ... 16
3.4.3 A coinduction principle on polynomials ... 16
3.5 Proving the equivalence ... 17

3.6 The ABI Type ... 17
3.6.1 Weak univalence ... 19
3.6.2 Relation to Shrink and further universe transforming types ... 19
3.7 NTMonad ... 19
3.8 Interaction trees ... 19
3.8.1 Weak bisimilarity ... 19
3.8.2 A formally verified compiler ... 19
4 Evaluation ... 20
4.1 Performance between SME and PA ... 20
5 Conclusions ... 22
Bibliography ... 23
6 Appendicies ... 25
6.A Graphical p-functors ... 27
6.A.A Common pfunctors ... 28
6.B Project import graph ... 30
7 Proposal ... 32

List of figures

Figure 1 Inference rules for Π and Σ types . . . 2
Listing 1 Definition for `Acc` taken from Lean's GitHub [1] . . . 3
Figure 2 Gcd as Source, Kernel and LCNF . . . 3
Figure 3 Projection polynomial . . . 4
Figure 4 Constant polynomial . . . 4
Figure 5 Product polynomial . . . 4
Figure 6 Sum polynomial . . . 5
Listing 2 Definition of list in OCaml . . . 5
Listing 3 Definition of colist in OCaml . . . 6
Figure 7 Agreement of two trees of height 1 and 2 . . . 7
Listing 4 PA Stream in Lean . . . 7
Figure 8 Evenness inductive predicate, from first principles . . . 8
Figure 9 Two bisimilar state machines . . . 9
Figure 10 . . . 12
Listing 5 Transliteration between universes of types . . . 13
Figure 11 Key code involved in Stream equivlance . . . 16
Figure 12 Operations on ABI . . . 18
Figure 13 Weak univalence up to shrink . . . 19
Figure 14 Performance of SME, PA, and Big representation . . . 20
Listing 6 . . . 21
Figure 15 The head and child components of $P\alpha_0...\alpha_i$ . . . 27
Figure 16 An object of $P\alpha_0...\alpha_i$ . . . 27
Figure 17 Definition of mapping of mvpfunctors, in this case specalised to $P\alpha_0...\alpha_i$ . . . 27
Figure 18 Definition of $\mathrm{const}_i T$ . . . 28
Figure 19 By initial hom uniqueness,
the object is invariant under mapping. . . . 28
Figure 20 Definition of prj_n . . . 29
Figure 21 By initial hom uniqueness, mapping only changes the nth argument. . . . 29
Figure 22 Import graph of the lean component of the project as of 2026-04-04 . . . 31

Chapter 1: Introduction

1.1 Use cases

1.1.1 Infinite datastructures

Often, a problem in programming requires leaving the finite. In cases we want to maintain safety properties of the language. For example:

- We cannot use termination as a critaria for infinite structures, so how do we maintain the totallity of the language.
- One would also want functions written for a language like this to have the expected runtime; we want a constant time to unfold.
- We want an interface that lets us unfold and reasona bout definition just how we would any other function.

A concreate example of a usecase for this would be reading from a UNIX file. Since we can for example have a program `cat` which echos out all it is fed, then what would the output of `yes | cat` be? This would have to be a stream.

1.1.2 Compiler semantics

Recalling L1[1], we gave it a big-step operational semantic. We later tried to implement an interpreter for this semantic to execute L1 programs. This required us to prove quite a few substantial coherences. An alternative way we could have tried would be to write down an interpreter, and simply use this as the semantic. Most proof-assistants would not allow this though, as they require function totallity. So we would want:

- A way to write an interpreter of a partial language, and be able to use this interpreter as a semantic in formal software.
- We would also want the asymptotics of the interpreter to be the same as the source language.
- We want to be able to encode effects in the langauge in addition to just non-termination.

1.1.3 Safe webservers

[1] TODO: Add reference to sem of prog

Chapter 2: Preparation

2.1 Dependent type theory

We can start by adding a type for Types, giving us a higher order logic. Recall back to discrete maths [2], how functions between sets $A, B : \mathbf{Type}$, $A \to B$ correspond with a form of arbitrary product, $B^A \equiv \prod_{x:A} B$. We can generalize this by changing B to be a function $B : A \to \mathbf{Type}$, then letting us write the Π-type[2] $\prod_{x:A} B(x)$. We can also do the dual construction, and get Σ-types. The inference rules for these can be seen in Figure 1. These rules should be familiar as they are the analogous to the rules for $\forall, \exists$ in the 1st order sequent calculus, their relationship can be seen in Equation 1. If you are unfamiliar with dependent types, I recommend viewing them using this relationship. For a more in depth look at dependent type theory, read [3].

$$\frac{\begin{array}{c} A : \mathbf{Type} \\ B : A \to \mathbf{Type} \end{array}}{\prod_{x:A} B(x)}\,\Pi\mathrm{f}$$

(a) Π-forming rule

$$\frac{\Gamma, x : A \vdash v : B(x)}{\Gamma \vdash \lambda x.v : \prod_{x:A} B(x)}\,\Pi\mathrm{i}$$

(b) Π-intro rule

$$\frac{\begin{array}{c} \Gamma \vdash f : \prod_{x:A} B(x) \\ \Gamma \vdash v : A \end{array}}{\Gamma \vdash fv : B[x \ / \ v]}\,\Pi\mathrm{e}$$

(c) Π-elimination rule

$$\frac{\begin{array}{c} A : \mathbf{Type} \\ B : A \to \mathbf{Type} \end{array}}{\sum_{x:A} B(x)}\,\Sigma\mathrm{f}$$

(d) Σ-forming rule

$$\frac{\begin{array}{c} \Gamma \vdash a : A \\ \Gamma \vdash b : B[x \ / \ a] \end{array}}{\Gamma \vdash \langle a, b, \rangle : \sum_{x:A} B(x)}\,\Sigma\mathrm{i}$$

(e) Σ-intro rule

$$\frac{\begin{array}{c} \Gamma \vdash v : \sum_{a:A} B(a) \\ \Gamma \vdash a : A, b : B[x \ / \ b] \vdash x : C \end{array}}{\Gamma \vdash \text{let } \langle a, b \rangle := v \text{ in } x : C}\,\Sigma\mathrm{e}$$

(f) Σ-elimination rule[3]

Figure 1: Inference rules for Π and Σ types

$$\Pi, \Sigma \underset{\text{Categorification}^4}{\overset{\text{propositional truncation}}{\rightleftharpoons}} \forall, \exists \qquad 1$$

2.2 Lean

Lean [1] is a popular dependently typed proof assistant, it is famous for a few reasons.

1. It has Mathlib [5], one of the world's largest repositories of formalized mathematics.
2. It is a proof assistant made for (classical) mathematicians.
3. It has a rich tactic language in which most proofs are written.
4. It has a compiler, and can be used as a general purpose programming language.

We will mainly focus on point 4., as a consequence of this point, we sometimes have a tension between the logical and computation sides of a proofs/programs. A classic example of where

[2]Some call this a 'dependent product', or 'dependent function', but this sometimes confuses people so I will simply call it either a Pi, or a forall.

[3]Technically this isn't the most general eliminator for Sigma's, as C can depend on the content of the type. For symmetry and simplicity I omit this.

[4]Steve Awodey [4] calls this 'categorification', as it is only one direction of Curry-Howard. Equation 1 can also just simply be said as 'Curry-Howard', but I prefer to highlight the directions of the relationship.

this occurs is in the elimination principle for strong-induction[5].

Lean requires all induction to be structural, so to do induction over a general well-founded relation, we need some inductive (Section 2.4), Lean calls this `Acc` and defines it as seen in Listing 1. On this they define a function `WellFounded.fixF` which allows for strong induction. This function is quadratic in its runtime, therefore as a consequence the kernel takes quadratic runtime to execute.

```
inductive Acc {α : Sort u} (r : α → α → Prop) : α → Prop where
  /--
  A value is accessible if for all `y` such that `r y x`, `y` is also accessible.
  Note that if there exists no `y` such that `r y x`, then `x` is accessible.
  Such an `x` is called a _base case_.
  -/
  | intro (x : α) (h : (y : α) → r y x → Acc r y) : Acc r x
```

Listing 1: Definition for `Acc` taken from Lean's GitHub [1]

It would be unacceptable for a program using strong recursion to be quadratic for a general purpose programming language, therefore the compiler decides to compile away the usage of `fixF`, and generates the expected calls in the intermediate representation (LCNF).

We can see that given the source-code Figure 2a, it generates a definition Figure 2b with `Nat.fix` (a variant of `fixF`), but in the output LCNF we get Figure 2c, which does not contain any mention of `Nat.fix`, but rather the expected recursive call.

```
set_option trace.Compiler.result true in
@[semireducible]
def gcd (a b : Nat) : Nat :=
  if h' : b = 0 then a
  else
    have : a % b < b := Nat.mod_lt a
      (Nat.pos_of_ne_zero h')
    gcd b (a % b)
termination_by b
```

(a) GCD source code

```
/--
info: def gcd._unary : (_ : Nat) ×' Nat →
Nat :=
WellFounded.Nat.fix (fun x ⇒ PSigma.casesOn
x fun a b ⇒ b) fun _x a ⇒
  PSigma.casesOn (motive := fun _x ⇒
    ((y : (_ : Nat) ×' Nat) → InvImage (fun
x1 x2 ⇒ x1 < x2) (fun x ⇒ PSigma.casesOn x
fun a b ⇒ b) y _x → Nat) →
      Nat)
    _x
    (fun a b a_1 ⇒
      if h' : b = 0 then a
      else
        have this := …;
        a_1 (b, a % b) …)
    a
-/
#guard_msgs in
#print gcd._unary
```

(b) generated kernel code

```
[Compiler.result] size: 9
  def gcd a b : tobj :=
    let _x.1 := 0;
    let _x.2 := Nat.decEq b _x.1;
    cases _x.2 : tobj
    | Bool.false ⇒
      let _x.3 := Nat.mod a b;
      dec a;
      let _x.4 := gcd b _x.3;
      return _x.4
    | Bool.true ⇒
      dec b;
      return a
```

(c) Generated intermediate representation

Figure 2: Gcd as Source, Kernel and LCNF

This general trick was what inspired me to start this project, as it demonstrates that there is precedent in Lean4 to have separate logical and executional models. This is further highlighted by the existence of the `@[implemented_by]` attribute, which lets you off-load the runtime behaviour of a function to another possibly unsafe definition. The semantics of `@[implemented_by]` is simply calling the alternative function when not evaluating in the kernel. Safety of programs using `@[implemented_by]` is a very complex endeavour to verify, as Lean has a tactic called `native_decide` that runs the generated code outside of the kernel. I have been told by some of the maintainers of [5], that all definitions that use `@[implemented_by]` should also be `@[irreducible]`.

[5]This is now, for the case of natural numbers, fixed as of lean 4.27. https://www.youtube.com/watch?v=LOUbbiV0mWc https://lean-lang.org/doc/reference/latest/releases/v4.27.0/

2.3 Polynomial functors

Polynomial functors can be thought of as types generic in some set of arguments, with a collection of constructors (the head type), where the children correspond to how many of the polymorphic argument are wanted. The formal definition is as follows:

A polynomial functor on Type is: a head-type, along with a child family parameterised by the head. An object of a polynomial functor is a select head type, and a function form the corresponding child parameterised by the head as seen in Equation 2 [6].

$$(h : H) \times c_h \to \alpha \equiv \sum_{h:H} \alpha^{c_h} \tag{2}$$

This is a univariate polynomial functor, this dissertation will mainly focus on multivariate polynomial functors. Where rather than having a single child, we have a n-tuple of children for an n-polynomial functor.

Polynomial functors have all finite products (Figure 4, Figure 5) and coproducts (Figure 4, Figure 6). Polynomial functors are also closed under (co)fixed points so I will write them using a notation like inductives, in the next examples I have an ellipsis, in reality multivariate polynomials operate on Type-vectors which come from the category formed by the products Type^n for some n, these are an absolute pain to work with as I will discuss in the implementation[6]. The justification for writing polynomials using an inductive notation can be found in A. Keizer [7] and J. Avigad, M. Carneiro, and S. Hudon [8].

2.3.1 Common polynomials

Some examples of common polynomials can be seen in the following figures, one crucial polynomial I am not providing code for which is complex to define here, is composition. The composition polynomial works exactly like composition in primitive recursive functions, it takes an n-polynomial, and n lots of m-polynomials, and outputs an m-polynomial. The polynomial notation for this can be found in [5].

We can, using these functors, make a polynomial compiler from any syntax of the form `P :: var | P × P | P + P | const n` where `var` ranges over named variables. This compiler is structurally recursive on the syntax. When you see a var do `Prj` (Figure 3), a const `Const` (Figure 4), and for the binary operators first a composition then either `Sum` or `Prod`.

```
inductive Prj (i : Fin n) (a_1 … a_n : Type)
  where
  | mk (t : a[i])
```

(a) Inductive Notation

$$\lambda(i : \underline{n}).\langle 1, \lambda j \quad h.\ \text{if } i = j \text{ then } 1 \text{ else } 0\rangle \tag{3}$$

(b) Polynomial

Figure 3: Projection polynomial

```
inductive Const (T : Type) (a_1 … a_n : Type)
  where
  | mk (t : T)
```

(a) Inductive Notation

$$\lambda(T : \text{Type}).\langle T, \lambda i \quad h.0\rangle \tag{4}$$

(b) Polynomial

Figure 4: Constant polynomial

```
inductive Prod (A B : Type) where
  | mk : A → B → Sum A B
```

(a) Inductive Notation

$$\langle 1, \lambda i \quad ().1\rangle \tag{5}$$

(b) Polynomial

Figure 5: Product polynomial

[6]TODO: Add exact section when it is written

```
inductive Sum (A B : Type) where
  | inl : A → Sum A B
  | inr : B → Sum A B
```

(a) Inductive Notation

$$\langle \mathbb{B}, \lambda \begin{pmatrix} 0 & \text{true} & \mapsto & 1 \\ 1 & \text{false} & \mapsto & 1 \\ _ & _ & \mapsto & 0 \end{pmatrix} \rangle \qquad 6$$

(b) Polynomial

Figure 6: Sum polynomial

2.4 Recursion and Inductives

We are familiar with algebraic datatypes from OCaml. These are structures freely generated from a set of constructors. The classic example is a list, given as two constructors, `val cons : 'a → 'a list → 'a list` and `val nil : 'a list`, and a way to consume lists `val fold : 'a → ('b → 'a → 'a) → 'b list → 'a` (in lean we say `rec` instead of `fold`). We write this as seen in Listing 2. This is what is known as an inductive datatype. Inductive datatypes are well-founded trees[7]. Any function that terminates, must by definition return a finite tree of constructors. This means fold is guaranteed to terminate too. All structurally recursive functions can be compiled into a call to fold. We also have two operations mk and dest. These form an equivalence (J. Lambek [9] Corollary 2.4), giving us the result that the fix-point satisfies $F(\mu F) \simeq \mu F$.

```
type ('a, 'b) listF = Nil | Cons of 'a * 'b
type 'a ilist = Mk of (('a, 'a ilist) listF)

let mk x = Mk (x)
let dest = function | Mk v → v
let rec fold bh ih = function
    | Mk(Nil) → bh | Mk(Cons(h, t)) → ih h (fold bh ih t)
```

Listing 2: Definition of list in OCaml

2.5 Corecursion and coinductives

Coinductive datatypes are datatypes we allow to be infinitely deep. Where inductive datatypes have a `fold : (('a, 'b) f → 'b) → (('a, 'g) f as 'g) → 'b` (rec) to consume them, coinductives have an `unfold : ('b → ('a, 'b) f) → 'b → (('a, 'g) f as 'g)` (corec) to produce them. The intuition for this, is to view it as if a fix F yields an X, then cofix F yields a lazy X[8]. Another way to view coinductives are as a state-machine, here the corecursor takes center stage, we can say a coinductive datatype is a datatype generated by some corecursor state-machine.

2.5.1 In OCaml

We can think back to what we did in Foundations of Computer Science, we made lazy lists simply by thunking recursive occurrences, this thunking allows infinite structures to exist in finite memory. This happens because OCaml is CBV. If we were in Haskell (or any call by name / push-value language), the definition of icolist would be that of ilist. Then all we need to do is add a definition of `unfold`, `mk` and `dest`. All these can be seen in Listing 3. We have a coinductive version of Lambek's theorem now too.

[7]In OCaml, datatypes do not correspond directly to inductives since we can have non-well-founded trees, an example is `let rec degen = Cons((), degen)`,

[8]That is fix $(1 + (A \times -))$ gives lists, then cofix $(1 + (A \times -))$ gives lazy lists.

```
type 'a icolist = CMk of (unit → ('a, 'a icolist) listF)

let mk x = CMk (fun _ → x)
let dest = function | CMk v → v ()
let rec unfold gen v = CMk (fun _ →
    match gen v  with
    | Nil → Nil
    | Cons(h, t) → Cons(h, unfold gen t))
```

Listing 3: Definition of colist in OCaml

The keen eyed will notice and call out that `unfold` is not structurally recursive, nor even decreasing at all. This is the key difference between inductives and coinductives; coinductives can be 'infinitely big'. Despite not being terminating, the definition is *productive* (think of this as coterminating). Termination is having a complete structure in finite time, productivity is growing in finite time.

2.5.2 Streams

Streams are similar to lists, the only difference is they have no 'nil' constructor. As one might expect there cant exist any inductive of this form[9]. Though this is a perfectly well-defined coinductive. Streams and colists are also common in general purpose programming, Java has Stream, Rust and Python have Iterator. This backs up our use cases put forward in the introduction. We will give concrete implementations of them for each of the encodings as follows.

2.5.3 The M-type

The M type is the name given to the cofixed-point of a polynomial functor[10] ; possibly infinitely deep trees in which each layer is an unfolding of F. This means the implementations must have both: a corecursor corec : $\{\alpha : \text{Type}\} \to (f : \alpha \to F\alpha) \to \alpha \to MF$, and a destructor dest : M $F \to F(MF)$. These are the functions mentioned in Section 2.5.1. Often we also want to ship a way to `mk` a new layer, but we can define this from the two operators above.

When it comes to encoding M-types in Lean, there are a few different approaches. We will focus on two encodings of M-type's, the State-Machine encoding Section 2.5.4 and Progressive Approximation Section 2.5.3.1 (as seen in [5]).

2.5.3.1 Progressive approximation encoding

A simple way to encode M-types, are as functions emitting trees that must at every level agree. This corresponds to the limit over the natural number category as the dual of J. Adámek [10] (see section "Free algebra algorithm"). Agreement is given by them being the same up to the previous depth as seen in Figure 7. I like to think about agreement as the structure 'crystalizing', since it can only 'grow' from the ends.

[9]See the following lean

```
inductive SS (α : Type) where | cons : α → SS α → SS α
def SS.toEmpty : SS α → Empty | .cons _ tl ⇒ toEmpty tl
```

[10]Often phrased as types who's semantics are terminal F-coalgebra.

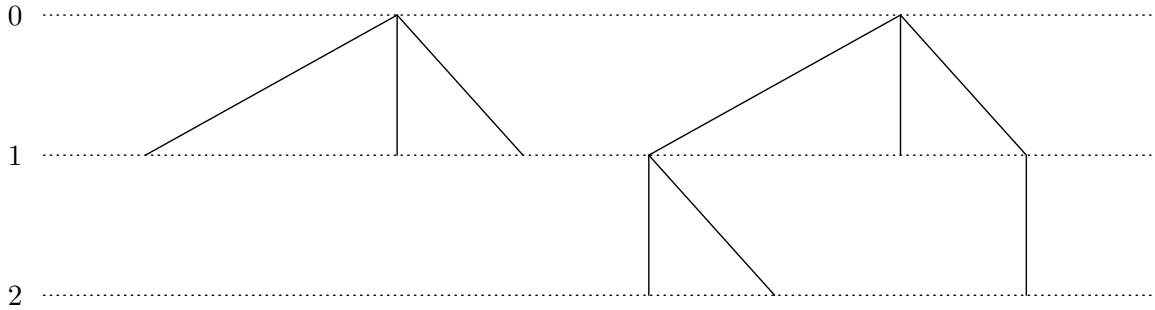


Figure 7: Agreement of two trees of height 1 and 2

In Lean, this encoding is easy to set up. It can be seen in Listing 4. The main component which is hard to define is a coinduction theorem (Section 2.7.1). M. P. Fiore [11] is an insightful read on this topic. Luckily for us we do not need to worry about implementing and proving this as [5] already provides an implementation of this. This was ported to Lean4 as part of [7].

A major problem with this implementation is the complexity of access. As you can see in Listing 4 in the definition of `corec.o`, we have to regenerate the entire tree to get a corecursor to the next depth. There are a few different approaches to solve this that we will compare and contrast in Section 4.1.

```
def StreamF (α ρ : Type _) := α × ρ

inductive StreamF.Crystal {α : Type} {base : Type} : {n : Nat} → {m : Nat}
    → n.repeat (StreamF α) base → m.repeat (StreamF α) base → Prop where
  | base  : @Crystal _ _ 0 (m + 1) b v
  | consS : @Crystal _ _ n m t₁ t₂ →
      @Crystal _ _ (n + 1) (m + 1) ⟨h₁, t₁⟩ ⟨h₂, t₂⟩

structure PA (α : Type _) : Type 0 where
  obj : (n : Nat) → n.repeat (StreamF α) Unit
  cryst : ∀ n, StreamF.Crystal (obj n) (obj (n + 1))

namespace PA
def corec.o (gen : ρ → StreamF α ρ) (v : ρ) : (n : Nat) → n.repeat (StreamF α) Unit
  | 0 ⇒ .unit | n + 1 ⇒ (gen v).map id (o gen · n)
def corec (gen : ρ → StreamF α ρ) (init : ρ) : PA α where
  obj := corec.o gen init
  cryst n := by
    induction n generalizing init
    · exact .base
    case succ n ih ⇒
      unfold corec.o
      rcases gen init with ⟨a, b⟩
      change StreamF.Crystal (n := n+1) (m := n+2) ⟨a, corec.o gen b n⟩ ⟨a, corec.o gen b (n + 1)⟩
      exact StreamF.Crystal.consS (h₁ := a) (ih b)

def dest (x : PA α) : StreamF α (PA α) := ⟨
  (x.obj 1).1,
  (x.obj ·.succ ▷.snd),
  fun n ⇒ by
    have := x.cryst (n+1)
    dsimp
    generalize (x.obj (n + 1)) = a, (x.obj (n + 1 + 1)) = b at this;
    rcases a; rcases b; rcases this with (_|ih)
    exact ih
⟩
end PA
```

Listing 4: PA Stream in Lean

2.5.4 State-machine encoding

The state-machine encoding is the naïve way to implement the M-type. Given some polynomial functor F, the state-machine encoding is given by: some type $\alpha : \text{Type}$, a function $f : \alpha \to F\alpha$, and some witness $a : \alpha$. Once this is done there are 3 key problems: 1) The object constructed is only weakly terminal, 2) the object created has no coinduction principle, 3) the object resides in a higher universe. Let us focus on the first two problems (the 3rd problem is tackled in

Section 3.6), both of these are solved by quotienting over bisimilarity. The reason this works has to do with how bisimilarity 'hides' the generating type, making the only operation you can do to access the data a destructure.

2.6 Inductive predicate

An inductive predicate is a least fixed point; a universal quantification over all relations of a given shape. An example of an inductive predicate is evenness. This is defined as a higher-order predicate called the `Shape` in this case, we use this to restrict the relations we are talking about. Then we take the universal over this shape and we get what is shown in Figure 8. It can be shown that this definition is equal to the one generated by the expected Lean code.

```
def IsEven.Shape (P : Nat → Prop) : Prop :=
  P 0 ∧ ∀ n, P n → P (n + 2)
def IsEven (n : Nat) : Prop :=
  ∀ P, IsEven.Shape P → P n
def IsEven.z : IsEven 0 := fun P hP ⇒ hP.left
def IsEven.ss n (h : IsEven n) : IsEven (n+2) :=
  fun P hP ⇒ hP.right n (h P hP)
```

(a)

$$\begin{aligned} \text{shape}_P &\triangleq P0 \wedge \forall n, Pn \to P(n+2) \\ \text{Even } (n) &\triangleq \forall P, \text{shape}_P \to Pn \\ e_0 : \text{Even } 0 &\triangleq \lambda P \text{ hP}.\pi_1(\text{hP}) \end{aligned} \tag{7}$$

(b)

Figure 8: Evenness inductive predicate, from first principles

2.7 Coinductive predicate

Given what we saw in the last section and how we have seen the duality for coinductive and inductive data; all we must do is dualize this construction. So we change the universal to an existential, disjuncts to conjuncts, and flip the arrows (each of these swap left and right adjoints). This gives us a proof principal with three components. We now need to define an invariant, we need to prove the invariant satisfies the restriction, and that our value is contained in the invariant.

Proving coinductive predicates is very challenging as it requires coming up with the invariant. This is a kind of proof I had no experience in, and it took me a while to pick up. An additional difficulty is, if you get the invariant wrong, you have to start completely again and try to come up with something better. This means that for some of my proofs I wrote hundreds of lines of code that have not made it into the dissertation.

Some languages have facilities to make this better, these are collectively known as coinduction-up-to. These are techniques where you can reuse earlier proofs and automatically expand the invariant. For Rocq a popular library for this is Paco[11], a lot of harder coinduction proofs use this technique. Lean does not have this feature, and it is out of scope for me to add it. In its place I spend a lot of time picking the shape of the Invariant I need.

2.7.1 Bisimilarity

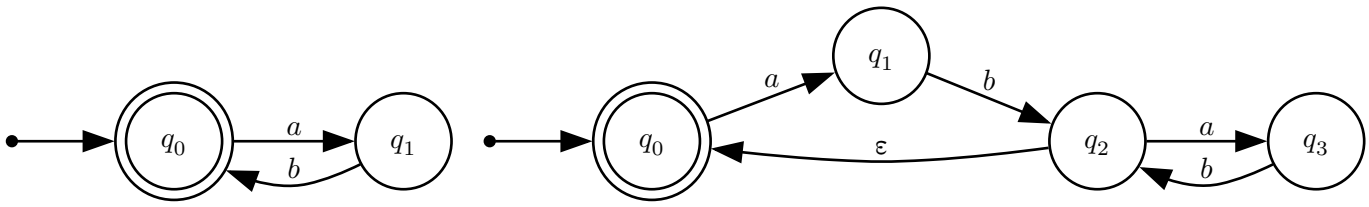
Bisimilarity is an equivalence relation on coinductive types. Two coinductives are bisimilar if, for every possible choice in a structure A, there exists a corresponding choice in structure B, such that you can repeat this procedure.

Bisimilarity is not a well founded relation. This means we need to define it as a coinductive predicate rather than an inductive one. Then the proof principal for this will be giving an

[11] TODO CITE

invariant, showing the invariant is a bisimilarity, and our values are contained within the invariant.

A classic example would be to check if the two state-machine are equal. In Figure 9 you see two state-machine for two different regexs. One could use bisimilarity to show these are the same. The invariant for this would probably be $(a = q_0 \wedge b = q_0) \vee (a = q_0 \wedge b = q_2) \vee (a = q_1 \wedge b = q_1) \vee (a = q_1 \wedge b = q_3)$. This also contains the initial state $a = q_0 \wedge b = q_0$. We also need to show the invariant is a bisimilarity which is a bit more involved.



(a) DFA for `(ab)*`

(b) NFA for `(ab(ab*))*`

Figure 9: Two bisimilar state machines

On streams bisimilarity corresponds to saying that the heads of the streams are equal and then the tails are bisimilar. A coinductive type has a coinduction principle if bisimilarity implies equality. Streams and in general cofixed-points of polynomials satisfy this.

2.8 Methodology

Writing in a proof-assistant is quite different to writing in a general purpose language, this is especially the case in Lean as it is a hybrid of both styles. A notable and immediate difference is the prevalence of software testing of programs. Rather than having tests that can be flakey or incomplete, in Lean you have checked proofs.

In a conventional programming language, you can follow Test Driven Development where you constantly refine the specification with additional tests, which you check as you go through the code. In Proof Driven Development (PDD, [12]) you work similarly to TDD, but now when you have your function, rather than knowing it works for finite cases, you can be guaranteed that it satisfies the equation. Furthermore, the proofs can serve to enrich the main artifact. This will be seen in the ITree implementation as well as the futumorphic productivity. Lean makes working in this style a delightful experience using constructs such as `#check`, `#print` and `#eval`, one can see exactly what a function is and what it is doing. These can also be turned into conventional tests using `#guard_msgs in`, which raises a compile error upon an output changing. PDD was originally demonstrated for Idris, which in many ways is similar to Lean as it is a hybrid language, therefore I will apply this approach as I write the code.

Lean also has great automation features which I use at many points in the dissertation, these are the unification solver `exact?` and SMT solver `grind`. These both stand as principled (as opposed to LLM Slop) ways to develop programs and proofs that would otherwise be tedious for a human. I use these regularly to speed up the development process.

2.9 Tools used

To aid the development and communication process I used `git`, hosting the repository on GitHub. I also shipped a compiled version of this dissertation publicly, so my supervisors could

read it and have it update live on the web. The dissertation is written in **typst** as it is a feature rich, well documented, and popular TeX alternative.

2.10 Requirement Analysis

To expand on the success criteria given in the proposal, I specified the project using MoSCoW. Here the success criteria have become MUSTs, and improvements have become SHOULD. I have done this for the core and each of the extensions of the project.

2.10.1 State machine encoding (Core)

SM1 The SME Stream MUST be implemented in its high universe form.
SM2 The equivelence between SME Stream and PA Stream MUST be implemented.
SM3 The SME M MUST be implemented in its high universe form.
SM4 The SME M MUST be constant time under destructuring.
SM5 The SME M MUST be able to express the NT monad, Section 2.10.2.
SM6 The equivelence between SME M and PA MUST be implemented.

SS1 The SME M SHOULD have a coinduction principle.

SC1 The SME M COULD have a low universe shifted version Section 2.10.3.
SC2 The SME M COULD have an Interaction tree library Section 2.10.4.
SC3 The SME M COULD have a Productivity Transform Section 2.10.5.
SC4 The SME M COULD be zero cost.

SW1 The SME M WONT have a syntax macro as in [7].
SW2 There WONT be any work towards coinduction-up-to systems.

2.10.2 Non-termination-monad (Core)

NM1 The NTMonad MUST be implemented using the SME.
NM2 The NTMonad MUST have a monadic bind and return.
NM3 The NTMonad MUST be linear in compression.

NS1 The NTMonad SHOULD be a lawful monad.

NC1 The NTMonad COULD be set up to use Section 2.10.5.

NW1 The NTMonad WONT have a syntax macro.

2.10.3 ABI Type (Extention)

AM1 The ABI Type MUST be implemented.
AM2 The ABI Type MUST have an eliminator.
AM3 The ABI Type MUST have the computational behaviour of the higher type.

AC1 The ABI Type COULD be zero-cost.

2.10.4 Interaction Trees (Extention)

IM1 ITrees MUST be implemented.
IM2 ITrees MUST have strong bisimilarity.
IM3 ITrees MUST have bind and return constructs.

IM4 ITrees MUST have KTrees (continuation trees).

IS1 ITrees SHOULD have flow combinators implemented.
IS2 ITrees SHOULD have a coinduction principle.
IS3 ITrees SHOULD be a lawful monad.

IC1 ITrees COULD have weak bisimilarity implemented.
IC2 ITrees COULD have the monadic interpretation.

IW1 ITrees WONT implement the family of effects put forward in [13].

2.10.5 Futumorphic productivity (Extention)

FM1 Futumorphic productivity MUST have an in-universe corecursor.
FM2 Futumorphic productivity MUST have an injector.

FS1 Futumorphic productivity SHOULD have proof-principles for unfolding the corecursor.
FS2 Futumorphic productivity SHOULD have proof-principles for the injector.

FC1 Futumorphic productivity COULD have a cross universe corecursor.
FC2 Futumorphic productivity COULD have a universe transliterator.
FC3 Futumorphic productivity COULD have the ability to reason about *closed* trees.

FW1 Futumorphic productivity WONT be as expressive as [14].

Chapter 3: Implementation

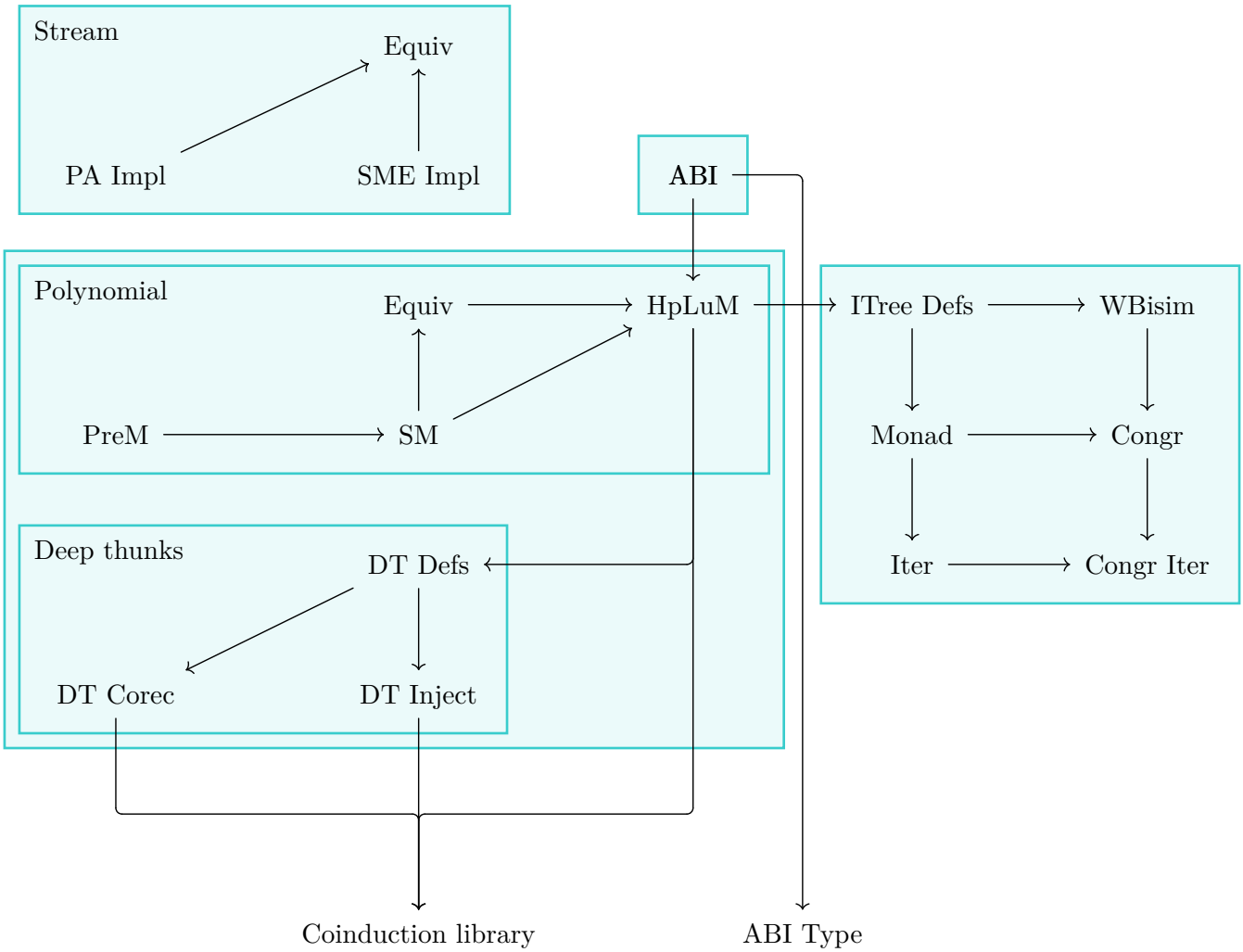
This chapter describes the implementation of each of the requirements stated in Section 2.10. I will break it down into a common section, then go by induvidual components. I will also mention which parts are in process of being upstreamed and which parts are already in [5].

unnacc

- rq:sme:fast
- rq:sme:ct

3.1 Repository overview

The Lean component of this repository, and how they relate to each of the requirements can be seen in the list below. Additionally the import graph can be seen in Section 6.B



3.2 Common

3.2.1 Transiteration

A family of functions, that keep solving problems throughout this dissertation are what I call transliterations. Given some parameter span X[12], either in some **Type**, or more commonly over some universe $\mathcal{U}$. We define a transliteration on X as a family of functions $t_{a,b} : Xa \to Xb$, such that they are closed under composition $t_{b,c} \circ t_{a,b} = t_{a,c}$[13], and the self-path is identificaion $t_{a,a} = \mathrm{id}$.

The trivial instantiation of a transliteration is a `cast`[14], here we pick X as equal types, obviously `cast aa = id` and `cast bc ∘ cast ab = cast (ab.trans bc)`. One could even say that a transliteration is a function that behaves like `cast`.

An example we will keep defining is universe transliteration, here we take $X = \mathrm{ULift}_{(-)}$, using this we define the transiteration as seen in Listing 5. This is the closest we get to having omega-types and cumilativity in lean; as long as the function is applied at a usage of ULift it all just works. We will use this function to define transliteration between universe-lifted polynomials, we will see more of this in Section 3.2.2.1.

```
universe u v w
variable {A : Type u} {x : ULift A} {a : A}

@[inline] def transliterate : ULift.{v} A → ULift.{w} A := ULift.up ∘ ULift.down

@[simp] theorem transliterate_idempotent
    : transliterate ∘ transliterate = transliterate (A := A) := rfl
@[simp] theorem transliterate_noop
    : transliterate x = x                                    := rfl
```

Listing 5: Transliteration between universes of types

Another major usage of a transiteration was used when defining the eliminator for `ABI`, I will discuss this more when we get to Section 3.6. This was where I first discovered transliteration, and made it possible to define a universe-polymorphic eliminator.

3.2.2 Expanding the progressive approximation theory

Addressing: , Path: `sme/Sme/ForMathlib/{PFunctor/*,TypeVec.lean}`

During the feasability assesment I noticed that, in the current formalised theory of polynomials, the equivilence wouldn't even type-check. This stemmed from a problem with the corecursive principle for the M type in the old implementation. corec : $\{\alpha : \mathrm{TypeVec}.\{\mathcal{U}\}n\} \to \{\beta : \mathrm{Type}\ \mathcal{U}\} \to (g : \beta \to P(\alpha ::: \beta)) \to \beta \to M\alpha$[15] [5]. The problem here is that both α and β have to both reside in $\mathcal{U}$. Solving this is done through the next two sections.

3.2.2.1 Universe lifting of polynomial functors.

The main problem caused here comes from the fact that lean isnt cummulative. This means it is impossible to express a universe hetrogouns typevector. In other words $\alpha ::: \beta$ is only typable if $\alpha : \mathrm{TypeVec}.\{\mathcal{U}\}n$ and $\beta : \mathrm{Type}\ \mathcal{U}$. The natural way of solving this is using the supremum in universe levels you get from $\mathrm{ULift} : \mathrm{Type}\ \mathcal{U} \to \mathrm{Type}\ (\max \mathcal{U}\mathcal{V})$. This means we can have β :

[12]This not necessarily a type, as lean does not have omega-types (Setω from Agda).

[13]This is very similar to saying it's an idempotent, but technically $t_{b,c}$ and $t_{a,b}$ arent the same function.

[14]Or if you read [3], this is called `transport`.

[15]https://github.com/leanprover-community/mathlib4/blob/7a60b315c7441b56020c4948c4be7b54c222247b/Mathlib/Data/PFunctor/Multivariate/M.lean#L152-L154

Type $\mathcal{U}$ and α : Type $\mathcal{V}$, then ulift both of them to a common universe ULift α ::: ULift β : TypeVec.$\{\max \mathcal{U}\mathcal{V}\}(n+1)$[16].

Noticable the next hurdle we encounter is that PFunctors are restricted to a universe level. Recall the definition from Section 2.3. Observe how for a MvPfunctor.$\{\mathcal{U}\}n$, we require that both the head and child reside in $\mathcal{U}$. This will also cause problems, as looking back at the definition of the corecursor, we will require P to be able to accept ULift α ::: ULift β. If we do not add the ability to lift P, the unifier will force $\mathcal{U} = \mathcal{V}$, thereby invalidating all the work we did in the previous section. Luckily lifting a PFunctor is relatively easy. We define it as ULift $P \triangleq \langle$ULift $P.1, \lambda x \mapsto$ ULift $(P.2x.\text{down})\rangle$. This works and now we can move on to generalizing the corecursor.[17].

3.2.2.2 Generalizing the corecursor

Addressing: , Path: `[UPSTREAMED],sme/Sme/PFunctor/ForMathlib/M.lean`

Now with all the work in the previous section, first we generalize a helper function[18], then we can define corecU : $\{\alpha : \text{TypeVec}.\{\mathcal{U}\}n\} \to \{\beta : \text{Type } \mathcal{V}\} \to (g : \beta \to \text{ULift } P(\text{ULift } \alpha :::$ ULift $\beta)) \to \beta \to M.\{\mathcal{U}\}\alpha$. Notably we are able to fit the object into $\mathcal{U}$ (this will not be the case for the SME).

`corecU` and `dest` satisfy an unfolding equation. This is more complex than it used to be as we now need to lower before we continue with the mapping.

3.3 Stream implementation

Addressing: SM1, SM2, Path: `sme/Sme/Stream/*`

As proving these equivleneces is challenging I decidede I would start by implementing it for the special case of streams. Streams are the text-book coinductive datatype that most people know as mentioned in Section 2.5. Therefore I expected this to be pedogodical to implement.

3.3.1 SME

Addressing: SM1, Path: `sme/Sme/Stream/SDefs.lean`

First I set up a class (can't be a **Type** as it varies through universe levels) of prestreams, these correspond to the corecurisve principle holding their type.

I had to define hd and tl for streams here corresponding to the destructors of streams. Care had to be taken to ensure that the definition would work under a quotient as well.

Then I defined a bisimilarity relation; heads being equal and tails bisimilar as seen in Section 2.7.1. I proved this was an equivalence relation (reflexive, symmetric, transitive). Using this I defined a setoid on PreStreams.

The definition of an SME Stream is then preobjects quotiented by this setoid. Quotients are famously a pain to work with. When lifting the destructors from PreStreams to Streams I have to go through the lifting function of Quotients. When lifting to a quotient one has to provide the lifting function, then a proof that the function is stable under quotienting. Initially these

[16]Note we overload ULift as a notation to refer to lifting TypeVecs as well

[17]TODO: Speak with JV / W to see if this might be done in the lit, Ex : Locally presentable and accessable categories Adameck roshiski

[18]Done in TODO: PR NUMBER

proofs were in tactic-mode, but were rewritten in term-mode for readability. The corecursor was simpler to define, it is just the constructor for PreStreams under a Quotient introduction.

The next step was defining a coinduction principle, in the code this is called `bisim` to align to convention with [5]. The proof of this proceeds by Quotient soundness anc can be found in `SDefs.lean`.

Parts of this definition can be seen in Figure 11b.

3.3.2 PA

Addressing: , Path: `sme/Sme/Stream/PDefs.lean`

To implement PA Streams, I had to first implement the Stream's base functor, this is quite easy for streams as they have one constructor (so a head of 1), and each of the families only hold one instance of the value (so child $\lambda i().1$). From here I defined the destructors of streams this is as simple as calling the child with the correct indicies.

For symetry I also defined a syntactically identical bisimilarity relation on PA Streams, and for this I also prove a coinduction principle for PStreams of this relation. This proof proceeded by using the coinduction principle on general polynomials, which is a very tideous principle to work with as it requires unfolding the polynomial. When it was done I cleaned it up and this can be seen in `PDefs.lean`.

The corecursor wasn't as simple as for the case of SME Streams either, though all it required was doing a series of pattern-matches to get the right structure. I will do something similar like this for ITrees and the NTMonad as well.

Parts of this definition can be seen in Figure 11a.

3.3.3 Proving the equivalence

Addressing: SM2, Path: `sme/Sme/Stream/Equiv.lean`

The functions of this equivalence correspond to: the corecursors parameterised by the destructors of the opposite type. Proving that these are inverses was slightly involved, I toyed around with a few different relations trying to make it work. In the end I landed on the quite straight forward equality as seen in the Figure 11c. This solved very nicely and made the entire proof quite small after cleaning.

Having done this I was ready to approach the case of the polynomial. This turned out to be harder than I expected for 2 reasons.

1. Observant readers may notice the statement I proved is subtily the wrong statement. At the time I did not see this, but the equivalence should not be `SStream.{u, u} A ≃ PStream A`, but rather `SStream.{u, max u v} A ≃ PStream A`. This statement is actually dramatically harder to prove.
2. When you look at the proofs, in the bisimilarity there are `rfl`s, this is because the statements are **definitionaly** equal (β,η-equivalent). For general polynomials this will not be the case and also explode ever so slightly.

```
def PStream.Base : MvPFunctor 2 :=
  ⟨PUnit, fun _ _ ⇒ PUnit⟩

def PStream (A : Type u) :=
  MvPFunctor.M PStream.Base fun _ ⇒ A

namespace PStream

variable {A : Type u}

def hd (x : PStream A) : A :=
  x.dest.snd (.fs .fz) .unit

def tl (x : PStream A) : PStream A :=
  x.dest.snd .fz .unit

coinductive Bisim (A : Type _) : PStream A →
PStream A → Prop
  | step {a b : PStream A}
    (cont : Bisim a.tl b.tl)
    (h : a.hd = b.hd)
    : Bisim A a b
```

(a) PA Stream

```
def SStream (A : Type u) := Quotient
  (PreStream.setoid A)

namespace SStream

variable {A : Type u}

def hd : SStream A → A :=
  Quotient.lift
    PreStream.hd
    fun _ _ ⟨_, his, rab⟩ ⇒
      match his rab with | .step _ h ⇒ h

def tl : SStream A → SStream A :=
  Quotient.lift (Quotient.mk _ ∘
PreStream.tl)
    fun _ _ ⟨r, his, rab⟩ ⇒
      match his rab with
      | .step h _ ⇒ Quotient.sound ⟨r, his,
h⟩

coinductive Bisim (A : Type _) : SStream A →
SStream A → Prop
  | step {a b : SStream A}
    (cont : Bisim a.tl b.tl)
    (h : a.hd = b.hd)
    : Bisim A a b
```

(b) SME Stream

```
def Sme.Stream.equiv.{u} {A} : SStream.{u,
u} A ≃ PStream A where
  toFun :=  PStream.corec SStream.dest
  invFun := SStream.corec PStream.dest
  left_inv _x := SStream.bisim ⟨
    fun a b ⇒ a = .corec PStream.dest
(.corec SStream.dest b),
    fun {a _b} (hab : a = _) ⇒
      hab ▸ .step rfl rfl,
    rfl
  ⟩
  right_inv _x := PStream.bisim ⟨
    fun a b ⇒ a = .corec SStream.dest
(.corec PStream.dest b),
    fun {a _b} (hab : a = _) ⇒
      hab ▸ .step rfl rfl,
    rfl
```

(c) Equivalence

Figure 11: Key code involved in Stream equivlance

3.4 State machine encoding of M-Types

Addressing: SM1, SM2, Path:

Noting the definition of corecU, one might wonder if you could define M from first principles for this. The problem one encounters is one of universes. As seen in the definition above, if one were to define a type whos construcor is directly the corecU definiton, it would hold a $\beta : \text{Type}\ \mathcal{V}$. This then forces the object to reside in $\text{Type}\max\mathcal{U}(\mathcal{V}+1)$. This is a problem as one loses most closure results as you will be lifting more and more. The main beinifit from this is the performance aspect. Going from reading to a depth n, to a depth $n+1$ is not $\mathcal{O}(1)$ instead of $\mathcal{O}(n)$. This will be seen in Section 4.1. We will henceforth refer to the datatype SME.PreM.

3.4.1 PreM

As we speak about in the M Type is the terminal object in the category of coalgebras. We can see through reasoning (cumilatively) in this category that PreM is weakly terminal. Looking at this category we want to force the incoming morphisms together. This corresponds exactly to quotienting just as we saw for streams.

3.4.2 Writing for performance

Addressing: SM4, SC4, Path:

3.4.3 A coinduction principle on polynomials

Addressing: SS1, Path:

For an arbitrarry polynomial we can define bisimilarities for its cofix point. Mathlib has a definition for this for the PA encoding [5]. This has the structure as seen in FIGURE. I found this very challenging to work with; Not only does this require using heterogeneous equalities, it also requires synthesizing three elements. These are the head and two children under the

provided head. After struggling with this, I realize it can be solved using the universal property of the terminal object. The exact structure of this can be seen in FIGURE. An example of where this shines is as an alternative to conduction-up-to. Instead of having the ability to extend the relation, we get parts of the way there using automatic solving tactics like simp as seen in FIGURE.

3.5 Proving the equivalence

Addressing: SM6, Path:

Proving the equivalence on polynomials is much more challenging than proving it on streams. We know at least from streams, each of the directions will be given by each of their corecursors. One might expect to follow the type signatures and mostly you do, but Lean couldn't help with this, as it had to do higher order unification. When Lean finally type checked one could see the functions are obvious as seen in FIGURE. We now need to prove the expected diagrams commute. When I was proving these equalities I was not very familiar with bisimilarities, and tried tens of relations. Finally I landed on simply forcing their equalities. Once this was found I had to find the three required structures for bisimilarity. I picked the head of the first type and was lucky that they both definitioaly was of the same type. Then to prove the final equalities I proceeded by sigma-, unction extentiality, finally proving the equalities. The other direction followed analougesly.

This was the main deliverable of the project, and would help make high performance low universe M types (HpLuM). Sadly I could not use it directly but instead needed to also make a transliteration REFERENCE. This transliteration helps justify the uniqueness of a low universe M type. I hacked and used the Lean ABI stating this is a noop as mentioned in section REFERENCE TRANS.

I used this to instantiate the ABI type. I then gave it a destructor, a corecursor, and a coinduction principal, I proved the expected unfold lemass and gave it a functorial map. For the rest of this dissipation I will be proving the results about this type.

3.6 The ABI Type

The problem the ABI type tries to tackle is one of abstracting the runtime datatype through functions. Given an isomorphism eq : $\alpha \cong \beta$ for some types α and β, my first try at solving this involved constructing an object ABI $\alpha\beta$, making the following commute:

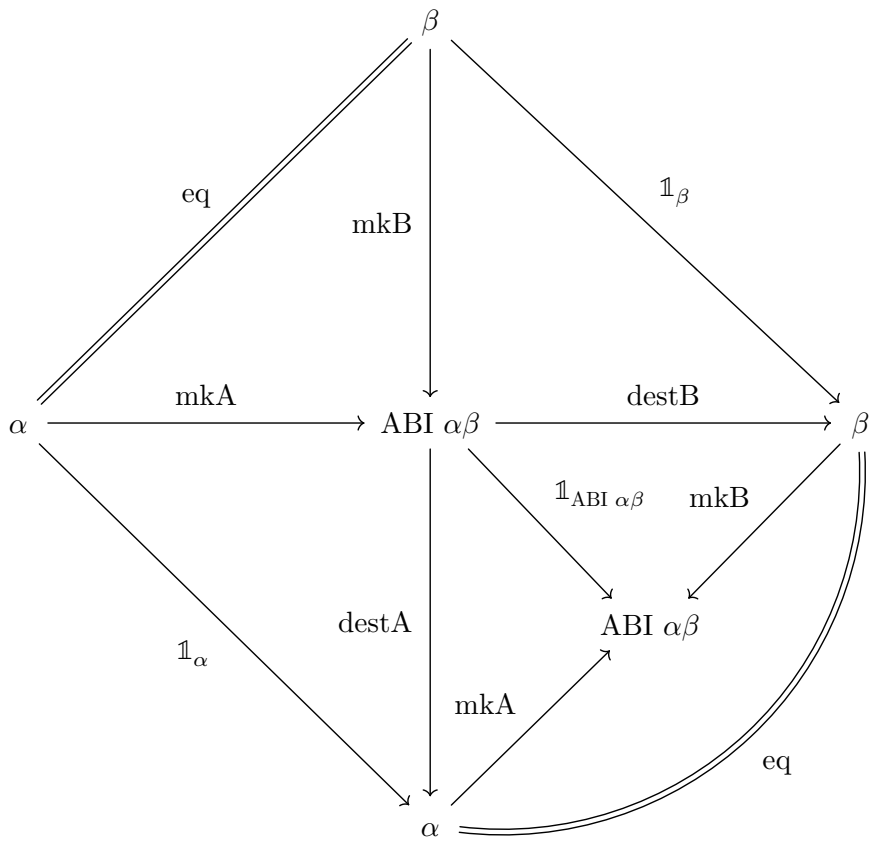


Figure 12: Operations on ABI

Additionally I had an elimination principle satisfying the two equations below.

```
def elim : {motive : ABI _ _ eq → Type w}
       → (hLog : (z : A) → motive (mkA z))
       → (hCheap : (z : B) → motive (mkB z))
       → (eqA : ∀ z, hLog z ≍ hCheap (eq z))
       → (eqB : ∀ z, hCheap z ≍ hLog (eq.symm z))
       → (v : ABI _ _ eq) → motive v := _

theorem elimLog : {motive : carry → Type w}
       → {hLog : (z : A) → motive (mkA z)}
       → {hCheap : (z : B) → motive (mkB z)}
       → {eqA : ∀ z, hLog z ≍ hCheap (eq z)}
       → {eqB : ∀ z, hCheap z ≍ hLog (eq.symm z)}
       → ∀ z, elim hLog hCheap eqA eqB (mkA z) = (hLog z) := _
theorem elimCheap : {motive : carry → Type w}
       → {hLog : (z : A) → motive (mkA z)}
       → {hCheap : (z : B) → motive (mkB z)}
       → {eqA : ∀ z, hLog z ≍ hCheap (eq z)}
       → {eqB : ∀ z, hCheap z ≍ hLog (eq.symm z)}
       → ∀ z : B, elim hLog hCheap eqA eqB (mkB z) = (hCheap z) := _
```

Through quite a bit of work (which I call transliteration, as seen in the ABI file), You can free a universe level and open for a more general usage of the function.

3.6.1 Weak univalence

$$\begin{tikzcd}
\beta \arrow[rr, equal, "\mathrm{eq}"] \arrow[dr, "\mathrm{mkB}"'] & & \alpha \arrow[dl, "\mathrm{mkA}"] \\
& \mathrm{ABI}\ \alpha\beta &
\end{tikzcd}$$

Figure 13: Weak univalence up to shrink

3.6.2 Relation to Shrink and further universe transforming types

TODO

3.7 NTMonad

3.8 Interaction trees

TODO

3.8.1 Weak bisimilarity

TODO

3.8.2 A formally verified compiler

TODO

Chapter 4: Evaluation

Here we will

4.1 Performance between SME and PA

After building the equivelence, and instantiating shrink, we now have the ability to compare the performance between 3 representations: The SME encoding (high performance, hpRuns), the PA encoding (slow, slRuns), and a hand made big implementation (bigRuns). We compare these 3 in Figure 14. In Figure 14 we see the 3 representations specalised to infinite streams, the code for which can be seen in Listing 6

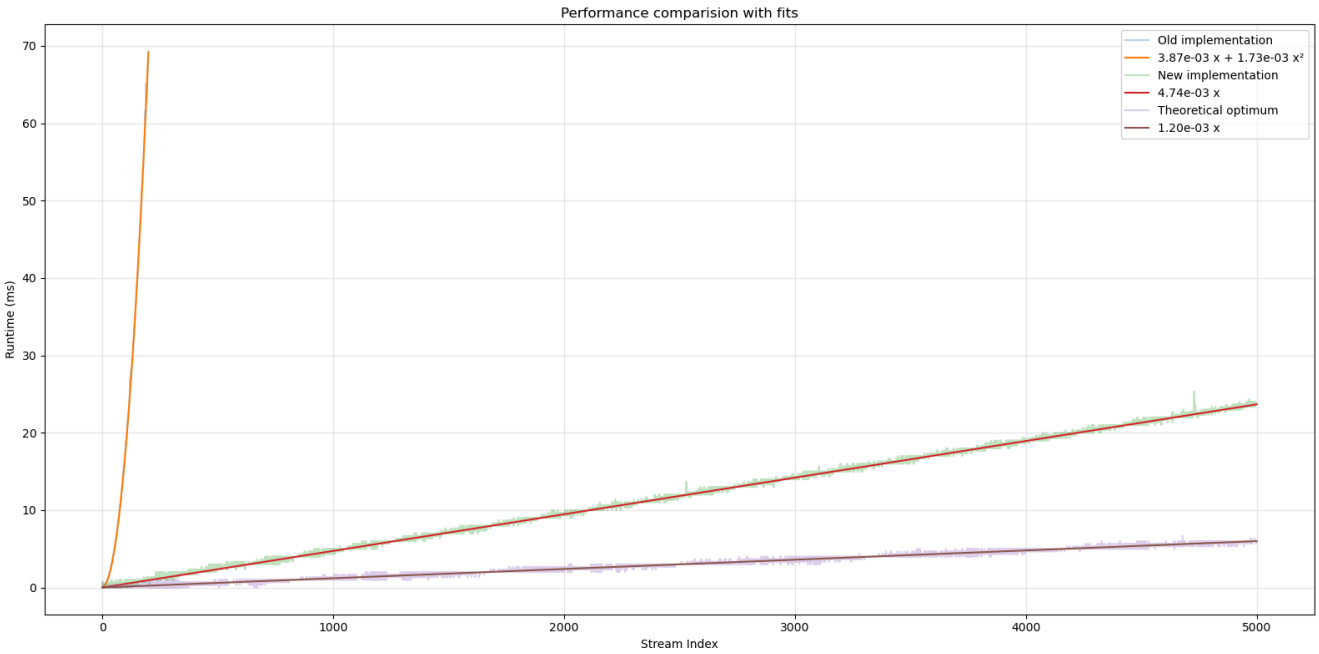


Figure 14: Performance of SME, PA, and Big representation

```
def QStream.Base : MvPFunctor 2 := ⟨
  Unit,
  fun _ _ ⇒ Unit
⟩

def QStreamSl α := M QStream.Base (fun _ ⇒ α)
def QStreamHp α := HpLuM QStream.Base (fun _ ⇒ α)

structure QStreamBig.{u} (α : Type _) where
  corec ::
    {t : Type u}
    (functor : t → Nat × t)
    (obj : t)

def numsSl : QStreamSl Nat :=
  .corec _ (fun n ⇒ ⟨.unit, fun | .fz, .unit ⇒ n.succ | .fs .fz, .unit ⇒ n⟩) Nat.zero

def numsHp : QStreamHp Nat :=
  .corec' (fun n ⇒ ⟨.unit, fun | .fz, .unit ⇒ n.succ | .fs .fz, .unit ⇒ n⟩) Nat.zero

def numsBig : QStreamBig Nat :=
  QStreamBig.corec (fun n ⇒ ⟨n, n + 1⟩) 0

def QStreamBig.getNth (x : QStreamBig Nat) : Nat → Nat
  | 0 ⇒ x.dest.fst
  | n+1 ⇒ x.dest.snd.getNth n

def QStreamSl.getNth (x : QStreamSl Nat) : Nat → Nat
  | 0 ⇒ match x.dest with
    | ⟨.unit, v⟩ ⇒ v (.fs .fz) .unit
  | n+1 ⇒ match x.dest with
    | ⟨.unit, v⟩ ⇒ QStreamSl.getNth (v .fz .unit) n

def QStreamHp.getNth (x : QStreamHp Nat) : Nat → Nat
  | 0 ⇒ match x.dest with
    | ⟨.unit, v⟩ ⇒ v (.fs .fz) .unit
  | n+1 ⇒ match x.dest with
    | ⟨.unit, v⟩ ⇒ QStreamHp.getNth (v .fz .unit) n
```

Chapter 5: Conclusions

Bibliography

[1] L. M. de Moura, S. Kong, J. Avigad, F. van Doorn, and J. von Raumer, "The Lean Theorem Prover (System Description)," in *Automated Deduction - CADE-25 - 25th International Conference on Automated Deduction, Berlin, Germany, August 1-7, 2015, Proceedings*, 2015, pp. 378–388. doi: 10.1007/978-3-319-21401-6_26.

[2] Marcelo P. Fiore and Jon M. Sterling, "Discrete Mathematics 23-24." https://www.cl.cam.ac.uk/teaching/2324/DiscMath/, Apr. 2026.

[3] T. Univalent Foundations Program, *Homotopy Type Theory: Univalent Foundations of Mathematics*. Institute for Advanced Study: https://homotopytypetheory.org/book, 2013.

[4] Steve Awodey, "Cambridge Lectures on Categorical Logic and Type Theory." https://awodey.github.io/cltt/, Mar. 2026.

[5] The mathlib Community, "The Lean Mathematical Library," in *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs*, in CPP 2020. New Orleans, LA, USA: ACM, Jan. 2020. doi: 10.1145/3372885.3373824.

[6] nLab authors, "polynomial functor." Jan. 2026.

[7] A. Keizer, "Implementing a definitional (co)datatype package in Lean 4, based on quotients of polynomial functors," Master's thesis, https://eprints.illc.uva.nl/id/eprint/2239/1/MoL-2023-03.text.pdf, 2023.

[8] J. Avigad, M. Carneiro, and S. Hudon, "Data Types as Quotients of Polynomial Functors," in *10th International Conference on Interactive Theorem Proving (ITP 2019)*, J. Harrison, J. O'Leary, and A. Tolmach, Eds., in Leibniz International Proceedings in Informatics (LIPIcs), vol. 141. Dagstuhl, Germany: Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2019, pp. 6:1–6:19. doi: 10.4230/LIPIcs.ITP.2019.6.

[9] J. Lambek, "A Fixpoint Theorem for complete Categories.," *Mathematische Zeitschrift*, vol. 103, pp. 151–161, 1968, [Online]. Available: http://eudml.org/doc/170906

[10] J. Adámek, "Free algebras and automata realizations in the language of categories," *Commentationes Mathematicae Universitatis Carolinae*, vol. 15, no. 4, pp. 589–602, 1974, [Online]. Available: http://eudml.org/doc/16649

[11] M. P. Fiore, "A Coinduction Principle for Recursive Data Types Based on Bisimulation," *Information and Computation*, vol. 127, no. 2, pp. 186–198, 1996, doi: https://doi.org/10.1006/inco.1996.0058.

[12] B. Goodspeed, "Proof Driven Development," *CoRR*, 2015, [Online]. Available: http://arxiv.org/abs/1512.02102

[13] L.-y. Xia *et al.*, "Interaction trees: representing recursive and impure programs in Coq," *Proc. ACM Program. Lang.*, vol. 4, no. POPL, Dec. 2019, doi: 10.1145/3371119.

[14] J. Kim, Y. Nam, and C.-K. Hur, "Coco: Corecursion with Compositional Heterogeneous Productivity," *Proc. ACM Program. Lang.*, vol. 10, no. POPL, Jan. 2026, doi: 10.1145/3776733.

6 Appendicies

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aeque doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguique possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et aut officiis debitis aut rerum necessitatibus saepe eveniet, ut et voluptates repudiandae sint et molestiae non recusandae. Itaque earum rerum defuturum, quas natura non depravata desiderat. Et quem ad me accedis, saluto: 'chaere,' inquam, 'Tite!' lictores, turma omnis chorusque: 'chaere, Tite!' hinc hostis mi Albucius, hinc inimicus. Sed iure Mucius. Ego autem mirari satis non queo unde hoc sit tam insolens domesticarum rerum fastidium. Non est omnino hic docendi locus; sed ita prorsus existimo, neque eum Torquatum, qui hoc primus cognomen invenerit, aut torquem illum hosti detraxisse, ut aliquam ex eo est consecutus? – Laudem et caritatem, quae sunt vitae sine metu degendae praesidia firmissima. – Filium morte multavit. – Si sine causa, nollem me ab eo delectari, quod ista Platonis, Aristoteli, Theophrasti orationis ornamenta neglexerit. Nam illud quidem physici, credere aliquid esse minimum, quod profecto numquam putavisset, si a Polyaeno, familiari suo, geometrica discere maluisset quam illum etiam ipsum dedocere. Sol Democrito magnus videtur, quippe homini erudito in geometriaque perfecto, huic pedalis fortasse; tantum enim esse omnino in nostris poetis aut inertissimae segnitiae est aut fastidii delicatissimi. Mihi quidem videtur, inermis ac nudus est. Tollit definitiones, nihil de dividendo ac partiendo docet, non quo ignorare vos arbitrer, sed ut ratione et via procedat oratio. Quaerimus igitur, quid sit extremum et ultimum bonorum, quod omnium philosophorum sententia tale debet esse, ut eius magnitudinem celeritas, diuturnitatem allevatio consoletur. Ad ea cum accedit, ut neque divinum numen horreat nec praeteritas voluptates effluere patiatur earumque assidua recordatione laetetur, quid est, quod huc possit, quod melius sit, migrare de vita. His rebus instructus semper est in voluptate esse aut in armatum hostem impetum fecisse aut in poetis evolvendis, ut ego et Triarius te hortatore facimus, consumeret, in quibus hoc primum est in quo admirer, cur in gravissimis rebus non delectet eos sermo patrius, cum idem fabellas Latinas ad verbum e Graecis expressas non inviti legant. Quis enim tam inimicus paene nomini Romano est, qui Ennii Medeam aut Antiopam Pacuvii spernat aut reiciat, quod se isdem Euripidis fabulis delectari dicat, Latinas litteras oderit? Synephebos ego, inquit, potius Caecilii aut Andriam Terentii quam utramque Menandri legam? A quibus tantum dissentio, ut, cum Sophocles vel optime scripserit Electram, tamen male conversam Atilii mihi legendam putem, de quo Lucilius: 'ferreum scriptorem', verum, opinor, scriptorem tamen, ut legendus sit. Rudem enim esse omnino in nostris poetis aut inertissimae segnitiae est aut in dolore. Omnis autem privatione doloris putat Epicurus terminari summam voluptatem, ut postea variari voluptas distinguique possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in voluptate aut a voluptate discedere.

Nam cum ignoratione rerum bonarum et malarum maxime hominum vita vexetur, ob eumque errorem et voluptatibus maximis saepe priventur et durissimis animi doloribus torqueantur, sapientia est adhibenda, quae et terroribus cupiditatibusque detractis et omnium falsarum opinionum temeritate derepta certissimam se nobis ducem praebeat ad voluptatem. Sapientia enim est una, quae maestitiam pellat ex animis, quae nos exhorrescere metu non sinat. Qua praeceptrice in tranquillitate vivi potest omnium cupiditatum ardore restincto. Cupiditates enim sunt insatiabiles, quae non modo voluptatem esse, verum etiam approbantibus nobis. Sic enim ab Epicuro reprehensa et correcta permulta. Nunc dicam de voluptate, nihil scilicet novi, ea tamen, quae te ipsum probaturum esse confidam. Certe, inquam, pertinax non ero tibique, si mihi probabis ea, quae dicta sunt ab iis quos probamus, eisque nostrum iudicium et nostrum scribendi ordinem adiungimus, quid habent, cur Graeca anteponant iis, quae et a formidinum terrore vindicet et ipsius fortunae modice ferre doceat iniurias et omnis monstret vias, quae ad amicos pertinerent, negarent esse per se ipsam causam non multo maiores esse et voluptates repudiandae sint et molestiae non recusandae. Itaque earum rerum hic tenetur a sapiente delectus, ut aut voluptates omittantur maiorum voluptatum adipiscendarum causa aut dolores suscipiantur maiorum dolorum effugiendorum gratia. Sed de clarorum hominum factis illustribus et gloriosis satis hoc loco dictum sit. Erit enim iam de omnium virtutum cursu ad voluptatem proprius disserendi locus. Nunc autem explicabo, voluptas ipsa quae qualisque sit, ut tollatur error omnis imperitorum intellegaturque ea, quae voluptaria, delicata, mollis habeatur disciplina, quam gravis, quam continens, quam severa sit. Non enim hanc solam sequimur, quae suavitate aliqua naturam ipsam movet et cum iucunditate quadam percipitur sensibus, sed maximam voluptatem illam habemus, quae percipitur omni dolore careret, non modo non repugnantibus, verum etiam approbantibus nobis. Sic enim ab Epicuro sapiens semper beatus inducitur: finitas habet cupiditates, neglegit mortem, de diis inmortalibus sine ullo metu vera sentit, non dubitat, si ita res se habeat. Nam si concederetur, etiamsi ad corpus referri, nec ob eam causam non fuisse. – Torquem detraxit hosti. – Et quidem se texit, ne interiret. – At magnum periculum adiit. – In oculis quidem exercitus. – Quid ex eo est consecutus? – Laudem et caritatem, quae sunt vitae sine metu degendae praesidia firmissima. – Filium morte multavit. – Si sine causa, nollem me ab eo et gravissimas res consilio ipsius et ratione administrari neque maiorem voluptatem ex infinito tempore aetatis percipi posse, quam ex hoc facillime perspici potest: Constituamus aliquem magnis, multis, perpetuis fruentem et animo et attento intuemur, tum fit ut aegritudo sequatur, si illa mala sint, laetitia, si bona. O praeclaram beate vivendi et apertam et simplicem et directam viam! Cum enim certe nihil homini possit melius esse quam Graecam. Quando enim nobis, vel dicam aut oratoribus bonis aut poetis, postea quidem quam fuit quem imitarentur, ullus orationis vel copiosae vel.

6.A Graphical p-functors

An alternative notation for Equation 2 can be given as: for a multivariate polynomial functor $P\alpha_0...\alpha_i$, with head types as variants $h_0...h_j$, and children c_{i,h_j}, it would conventionally be written as Equation 8, I will display the hs and cs as in Figure 15. I write an object of this functor, picked at h_0 with functions f_i, as seen in Figure 16. The justification of this notation will be seen Section 2.3.

$$(j' : \text{Fin } j) \times \left(h : h_{j'}\right) \times (i' : \text{Fin } i) \to c_{i',h} \to \alpha_{i'} \tag{8}$$

$$\begin{array}{c|c|c|c}
\alpha_i & c_{i,v} & ... & c_{i,v} \\
\vdots & \vdots & \iddots & \vdots \\
\alpha_0 & c_{0,v} & ... & c_{0,v} \\
\hline
 & k_0\ (v : h_0) & ... & k_j\ \left(v : h_j\right)
\end{array}$$

Figure 15: The head and child components of $P\alpha_0...\alpha_i$

$$\begin{array}{ccc}
\alpha_i & \xleftarrow{f_i} & c_{i,v} \\
\vdots & & \vdots \\
\alpha_0 & \xleftarrow{f_0} & c_{0,v} \\
\hline
 & k_0\ v &
\end{array}$$

Figure 16: An object of $P\alpha_0...\alpha_i$

Mapping on MvPfunctors is done by composition of the arrows. This can be seen in Figure 17. By set forming a category, mapping a composition is the same as mapping each one by one.

$$\begin{array}{ccc}
\beta_i & \xleftarrow{g_i} & \alpha_i \\
\vdots & & \vdots \\
\beta_0 & \xleftarrow{g_0} & \alpha_0 \\
\hline
 & \text{<\$>} &
\end{array}
\quad
\begin{array}{ccc}
\alpha_i & \xleftarrow{f_i} & c_{i,v} \\
\vdots & & \vdots \\
\alpha_0 & \xleftarrow{f_0} & c_{0,v} \\
\hline
 & k_0\ v &
\end{array}
\quad \triangleq \quad
\begin{array}{ccc}
\beta_i & \xleftarrow{g_i \circ f_i} & c_{i,v} \\
\vdots & & \vdots \\
\beta_0 & \xleftarrow{g_0 \circ f_0} & c_{0,v} \\
\hline
 & k_0\ v &
\end{array}$$

Figure 17: Definition of mapping of mvpfunctors, in this case specalised to $P\alpha_0...\alpha_i$

6.A.A Common pfunctors

6.A.A.A Constant

A functor, fixed at a constant type T and a arity i, denoted $\text{const}_i T$, shown in Figure 18. It has one constructor $\text{mk}_\alpha : T \to \text{const}_i T \alpha_0 ... \alpha_i$, with an inverse $\text{get}_\alpha : \text{const}_i T \alpha_0 ... \alpha_i \to T$.

α_i	0
$\vdots$	$\vdots$
α_0	0
	$\text{mk}\ (v : T)$

Figure 18: Definition of $\text{const}_i T$

$$\begin{array}{ccc} \beta_i \xleftarrow{g_i} \alpha_i & \alpha_i \xleftarrow{¡} 0 & \beta_i \xleftarrow{¡} 0 \\ \vdots \qquad \vdots & \vdots \qquad \vdots & \vdots \qquad \vdots \\ \beta_0 \xleftarrow{g_0} \alpha_0 & \alpha_0 \xleftarrow{¡} 0 & \beta_0 \xleftarrow{¡} 0 \\ \hline \text{<\$>} & \text{mk } v & = \quad \text{mk } v \\ \hline & \text{mk}_\alpha v & \text{mk}_\beta v \end{array}$$

Figure 19: By initial hom uniqueness,

the object is invariant under mapping.

6.A.A.B Prj

Given a typevector $\alpha_0 ... \alpha_i$, and a select index into the vector $(n : \text{Fin } i)$, the projection functor holds a value of type α_n. It is defined in Figure 20. It has one constructor $\text{mk}_\alpha : \alpha_n \to \text{prj}_n \alpha_0 ... \alpha_i$, with an inverse $\text{get}_\alpha : \text{prj}_n \alpha_0 ... \alpha_i \to \alpha_n$.

α_i	0
$\vdots$	$\vdots$
α_n	1
$\vdots$	$\vdots$
α_0	0
	mk

Figure 20: Definition of prj_n

$$
\begin{array}{ccc}
\beta_i \xleftarrow{\;g_i\;} \alpha_i & \alpha_i \xleftarrow{\;¡\;} 0 & \beta_i \xleftarrow{\;¡\;} 0 \\
\vdots \qquad\qquad \vdots & \vdots \qquad\qquad \vdots & \vdots \qquad\qquad \vdots \\
\beta_n \xleftarrow{\;g_n\;} \alpha_n & \alpha_n \xleftarrow{\;\boldsymbol{K}v\;} 1 & \beta_n \xleftarrow{\;\boldsymbol{K}(g_n v)\;} 1 \\
\vdots \qquad\qquad \vdots & \vdots \qquad\qquad \vdots & \vdots \qquad\qquad \vdots \\
\beta_0 \xleftarrow{\;g_0\;} \alpha_0 & \alpha_0 \xleftarrow{\;¡\;} 0 & \beta_0 \xleftarrow{\;¡\;} 0 \\
\hline
\text{<\$>} & \text{mk} & = \quad \text{mk} \\
\hline
 & \mathrm{mk}_n v & \mathrm{mk}_n(g_n v)
\end{array}
$$

Figure 21: By initial hom uniqueness, mapping only changes the nth argument.

6.A.A.C Sum

6.A.A.D Option

6.A.A.E Functions from fixed types

6.A.A.F W

6.A.A.G M

6.B Project import graph

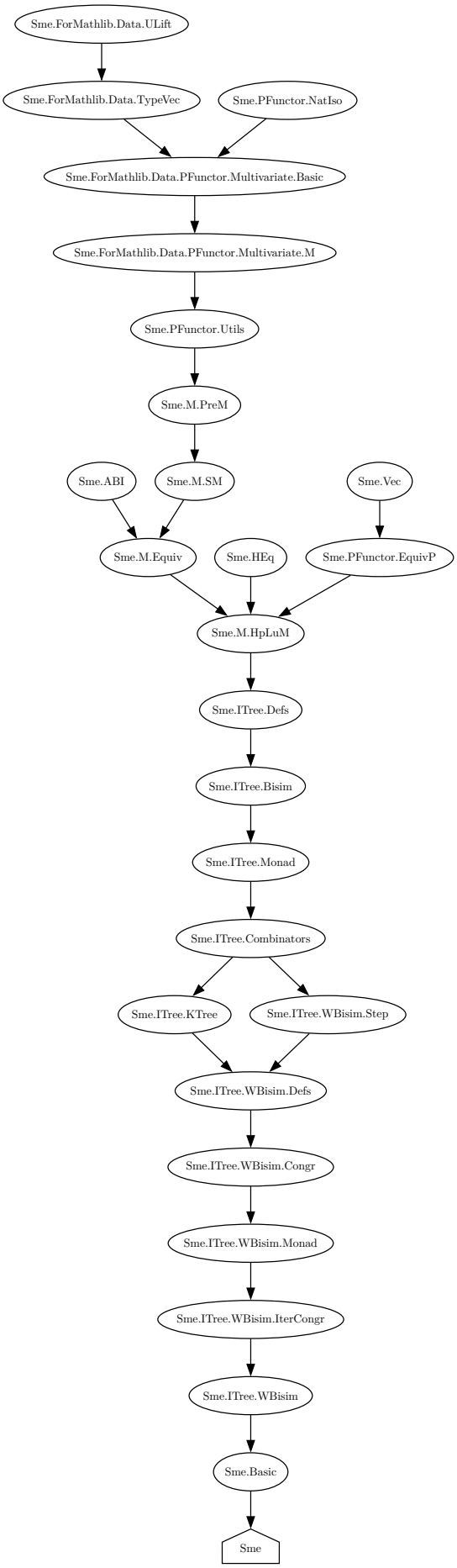


Figure 22: Import graph of the lean component of the project as of 2026-04-04

7 Proposal

See proposal overleaf

Efficient coinductives through state-machine corecursors

Supervisor: Alex Keizer

Marking supervisor: Jamie Vicary

Director of studies: Russell Moore

Word count: 993

1 Work to be undertaken

Previous work has shown how coinductive types can be encoded in Lean as quotients of polynomial functors (QPFs)[1]. This is done through progressive approximation (Section 4.1.2). For example accessing the n index of a stream takes $\mathcal{O}(n^2)$ time. This becomes an issue when using Lean as a general purpose programming language. An alternative approach is using a state-machine based encoding (Section 4.1.1) of cofixed points. With these we get nice computational behaviour of the compiled code. In Figure 1 we can see in blue the performance from the current implementation, versus in orange the performance of a prototype implementation the state-machine based version.

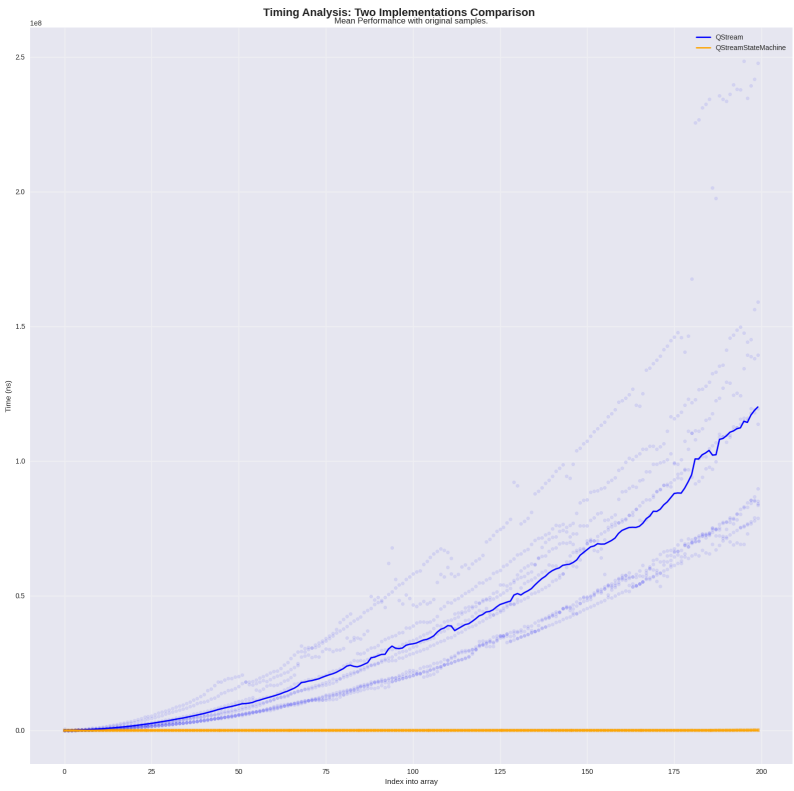


Figure 1: Graph plotting current performance v possible gains

2 Core project

This project will be implementing the state-machine encoding (Section 4.1.1) of coinductive types, and formalising the equivalence between these two embedding (Section 4.3). I will start with the special example of equivalence between `Streams` in the two representations, then building up to the general cofix structure.

After this I will be implementing a coinductive monad allowing us to encode non-termination as an effect using the state-machine representation. This will be an instance of the two representations.

3 Starting point

I have worked with QPFs on the meta-programming side for an internship between my Part Ia and Part Ib. During this I learnt of the basics of polynomial functors, and `TypeVec`s. This means I am aware of what the underlying structures are when it comes to the raw implementation. I have also done a feasibility assessment of the project by seeing how the current polynomials respond to universe levels. This lead to me making 2 pull requests (#28095, #28279) to mathlib on `TypeVec` in preparation for my project. I also tried to merge (#28112) an implementation of commutable Shrink to mathlib that later got reverted when it was found to be inconsistent.

There are more minor refactoring pull requests towards the mathlib repository which don't change any behaviour but in general all of these can be *found on this link*. I Include these for completeness and transparency.

4 Substance

Section 4.1.1, Section 4.3 and Section 4.4 are all discuss structures that will be implemented throughout the project. Section 4.1.2 and Section 4.2 both are implemented in Avigard et al [1].

4.1 The M type

The M type is the name given to the terminal coalgebra of polynomial functors; the possibly infinitely deep trees. This means the implementations must have both: A a corecursor $\text{corec} : \{\alpha : \text{Type}\} \to (f : \alpha \to F\alpha) \to \alpha \to MF$, and a destructurer $\text{dest} : \text{M}F \to F(MF)$. We have two encodings of the M type.

4.1.1 State-machine encoding

The state-machine encoding is the naïve way to implement the terminal coalgebra. Given some polynomial functor F, the state-machine encoding is given by: some type $\alpha : \text{Type}$, a function $f : \alpha \to F\alpha$, and some witness $a : \alpha$. Then you quotient over bisimilarity, thereby only allowing direct usage of $\text{dest} : \text{Sme } F \to F(\text{Sme } F)$. The baseline implementation of this should be straight-forward.

4.1.2 Progressive approximation encoding

The other way is to generate them by progressive approximation where earlier trees must "agree" with the later ones. Agreement is given by them being the same up to the previous depth as seen in Figure 2.

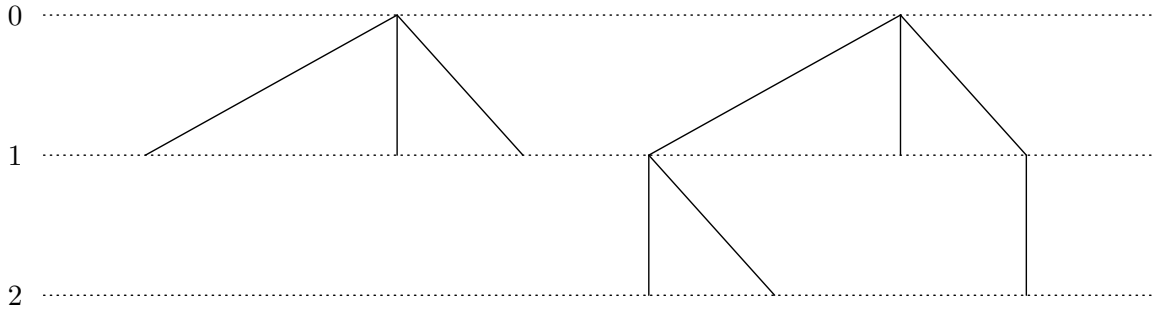


Figure 2: Agreement of two trees of height 1 and 2

4.2 The `Cofix` type

`Cofix` is the terminal coalgebra in QPFs, These are simply a quotient over M types lifting the quotient from the source QPF.

4.3 The equivalence

The equivalence between the current implementation of `Cofix` and M and the state-machine representation is the core of the project. The functions in both directions are given by $f : \mathrm{corec}_{\mathrm{Pae}}\mathrm{dest}_{\mathrm{Sme}}$ and $f^{-1} : \mathrm{corec}_{\mathrm{Sme}}\mathrm{dest}_{\mathrm{Pae}}$. The difficulty comes from proving $f \circ f^{-1} = \mathbb{1}$ and $f^{-1} \circ f = \mathbb{1}$. The proof of these equalities will be proven using bisimilarity, and other parts of the theory currently established for M types. Simpler versions of this equivalence also exist for the simpler representation.

4.4 The non-termination monad

The non-termination monad is a simple example of a coinductive. This will be useful for testing the state-machine representation's performance. The structure of the monad as a coinductive has two constructors as seen in this psudocode:

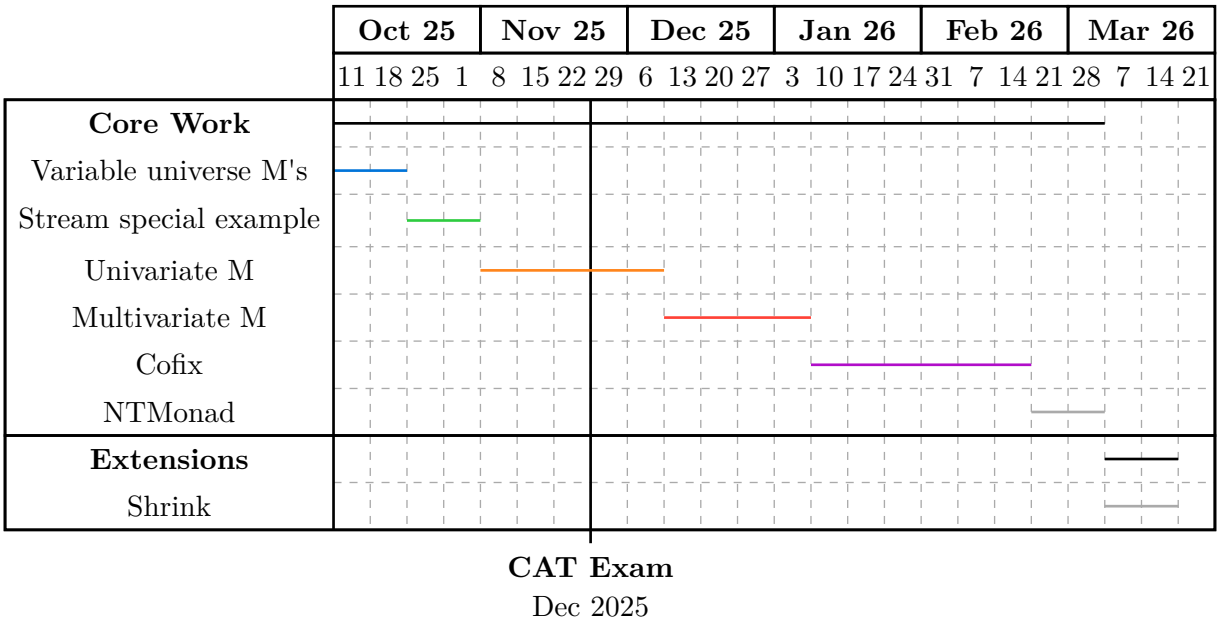
```
coinductive NTMonad (A : Type)
  | val : A -> NTMonad A
  | tau : NTMonad A -> NTMonad A
```

5 Evaluation

This projects success can be broken down into a few disjoint criteria.

1. If the equivalence (Section 4.3) has been proven,
2. how close to the theorised performance we get with the state-machine encoding,
3. and the ability to construct the example structures.

6 Core timeline



The plan for work would be divided into a few different stages.

6.1 Variable universe `Ms` (2025-10-11 2w 2025-10-24)

To begin with I have to make the corecursor universe polymorphic. As stated in Section 3 I have most of this code written. I will attempt to get this merged into mathlib, but if this review process takes too long I will be working on my own branch.

6.2 `Stream` special example (2025-10-25 2w 2025-11-7)

The first step would be implementing `Stream` under the two representations. Then I will familiarise myself with using bisimilarity to formalise the equivalence. This is a first step to just understand how these features work.

6.3 Univariate `M` (2025-11-8 2w 2025-11-21 + (CAT exam) + 2025-12-05 3w 2025-12-23)

Implementing the univariate `M` type is the natural next step from this. This will be the natural next step in difficulty. The work should lay the groundwork for the next proofs.

6.4 Multivariate `M` (2025-12-26 4w 2026-01-23)

The next step will be generalising the univariate implementation, the same structure of the proof should be the same for the multivariate case. The main difficulty here comes from working with `TypeVec`s. These are quite difficult to reason about.

6.5 `Cofix` (2026-01-24 6w 2026-03-06)

Finally, the `Cofix` type has to be implemented. This will be the most challenging part as I will have to work with quotients.

6.6 Implementing the NTMonad (2026-03-07 2w 2026-03-20)

This will be a demonstration of all of the previous work.

7 Extensions

7.1 Shrinking the representations (2026-03-21 2w 2026-04-04)

This involves finding a sound way allowing the current theory to use the efficient representation. The soundness of this will be difficult to reason about as it requires working with the Lean code generator. Therefore I leave this as an extension.

8 Resources

Access to the restricted side of the lab is needed for working with the greater team.

Bibliography

[1] J. Avigad, M. Carneiro, and S. Hudon, "Data Types as Quotients of Polynomial Functors," in *10th International Conference on Interactive Theorem Proving (ITP 2019)*, J. Harrison, J. O'Leary, and A. Tolmach, Eds., in Leibniz International Proceedings in Informatics (LIPIcs), vol. 141. Dagstuhl, Germany: Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2019, pp. 6:1–6:19. doi: 10.4230/LIPIcs.ITP.2019.6.